CheckList Seminar — Speaking Script

> **AI-generated content (Claude).** Based on Ribeiro et al. (2020), *Beyond Accuracy: Behavioral Testing of NLP Models with CheckList*, ACL 2020. Review before presenting.

Slides: https://claude.ai/artifact/H2kUtV9vqwkdLNNRsoDxKc · **Paper:** https://aclanthology.org/2020.acl-main.442.pdf

Part	Slides	Topic	Time
Opening	1	Cover	30 s
Part 1	2 · 3 · 4	Motivation · Contributions · Related work	~4 min
Part 2	5 · 6 · 7	Key concepts	~3 min
Part 3	8 · 9 · 10	Methodology	~3 min
Part 4	11 · 12 · 13	Results · Applications · Conclusion	~4 min
Closing	14	References	15 s

Words in **bold** are the points that must not be skipped.

Slide 1 — Cover

> “Good morning, everyone. Today we’re presenting **Beyond Accuracy: Behavioral Testing of NLP Models with CheckList**, by Ribeiro, Wu, Guestrin and Singh, published at ACL 2020.
>
> The question behind this paper is simple: if a language model scores over 90% on a benchmark, does that mean it really understands language? The authors show that the answer is often no, and they propose a way to find out, borrowed from software testing.
>
> Our talk has four parts: the motivation, contributions and related work; the key concepts; the methodology; and the results, applications and conclusion.”

Part 1 — Motivation, contributions and related work

Slide 2 — The problem: accuracy hides failures

> “Let me start with the problem behind this paper.

> Today, language models are usually judged by **one number**: their accuracy on a test set. The authors see three problems with that.
>
> First, **biased test sets**: the test set has the same biases as the training data, so a model can look good in the test and still fail in the real world.
>
> Second, **one number**: a single score doesn't tell us *where* the model fails, or how to fix it.
>
> And third, **narrow alternatives**: other evaluations existed, like robustness or fairness tests, but each one looks at only one task or one skill.
>
> Here's an example from the paper. BERT gets **92.7%** on the SST-2 sentiment benchmark, which looks excellent. But on sentences like 'I thought the plane would be awful, but it wasn't' — which is actually positive — it fails **85%** of the time.
>
> So a great score can hide very basic mistakes. The authors' answer is to test language models the way we test software, one skill at a time. That's CheckList, and these are its contributions."

Tip: read the example slowly and stress that the sentence is positive.

Slide 3 — What the paper contributes

> "So what does the paper propose? **Three things.**
>
> First, **a method**: test language models the way we test software, checking **one skill at a time** — like negation or names — instead of looking at one overall score. It works for any task and any model.
>
> Second, **a free tool** that generates **thousands of tests automatically**, so this kind of testing is cheap.
>
> And third, **evidence** that it works: the authors found serious bugs both in research models like BERT and in commercial products from Microsoft, Google and Amazon. And people who used CheckList found about **three times as many bugs** as people who didn't.
>
> Now, how does this compare with previous work?"

Slide 4 — Related work

> "Each earlier approach covers only **one piece** of the problem.
>
> **Held-out accuracy**, like the SST-2 or QQP leaderboards, gives one number but doesn't show where the model fails. CheckList gives a score per capability.
>
> **Robustness tests**, with typos, noise or adversarial examples, look at only one skill. In CheckList, robustness is just one row of the matrix.

Fairness and consistency tests, like swapping names, are built for one task at a time. CheckList reuses the same tests across tasks.

Challenge sets are hand-built, small, artificial, and only test hard cases. CheckList uses templates and real data, and tests cases that should be easy.

Probes look inside the model. For example, a probe shows BERT can recognize names with 96.7% accuracy, but BERT fine-tuned on a real task still fails simple tests with names. Knowing something inside doesn't mean the model uses it.

And **software testing** is not a competitor — it's the inspiration. CheckList's first test type is like a unit test, and the other two are metamorphic tests.

The authors are also clear about what CheckList does not cover: labelling errors, data versioning, security attacks and interpretability.

Now, to understand how CheckList works, we need some key concepts."

Part 2 — Key concepts

Slide 5 — From accuracy to behavioral testing

"Thank you. I'll start with two ideas.

The first is **held-out accuracy**: the percentage of correct answers on examples the model never saw during training. It's the standard way to evaluate models, but it puts everything into one number.

The second is **behavioral testing**, also called black-box testing. We don't look inside the model; we only give it inputs and check the outputs, and each test checks one specific behavior.

A good way to picture it: accuracy is a student's final average; behavioral testing is the **report card per subject**. A student can have a great average and still be failing one subject completely. That's exactly what happens with these models."

Slide 6 — Capabilities, perturbations, metamorphic tests

"Three more terms you'll hear throughout the paper.

A **capability** is a skill the task needs. For example, negation: the model has to understand that 'not poor' is not negative.

A **perturbation** is a small, controlled change to the input, for example replacing 'Chicago' with 'Dallas' in a sentence.

A **metamorphic test** compares outputs instead of checking them against an answer key. If I only change the city name, the sentiment should stay the same. I don't need to know the correct sentiment — I just check that it didn't change.

This matters a lot, because it means we can test models on **real data that has no labels**."

Slide 7 — The tasks and models

"Finally, the tasks and models in the paper.

There are three tasks. **Sentiment analysis**: is a text positive, negative or neutral? **Quora Question Pairs**: do two questions ask the same thing? And **reading comprehension**: answering a question from a paragraph, using the SQuAD dataset.

The models are mainly **BERT and RoBERTa**: Transformer language models pretrained on huge amounts of text and then fine-tuned for each task.

One more term: a **masked language model** fills in a hidden word. Given 'I really [mask] the flight', it might suggest 'enjoyed' or 'hated'. CheckList uses this to help generate tests.

With these concepts in place, let's see how the method works."

Part 3 — Methodology

Slide 8 — A matrix of capabilities × test types

"Thanks. CheckList organizes testing as a **matrix**. The rows are capabilities, like vocabulary, named entities and negation. The columns are test types, which I'll explain next. Each cell shows a **failure rate**: the percentage of test cases the model got wrong.

These numbers come from a real commercial sentiment model in the paper. Look at negation: it fails **76.4%** of a very simple test.

And this is all black-box: no access to the model's internals is needed, which is why the authors could test commercial APIs."

Slide 9 — Three test types

"These are the three columns of the matrix.

MFT, a Minimum Functionality Test, is like a unit test: simple examples with a known answer. 'I didn't love the flight' should be negative.

INV, an Invariance test, applies a change that shouldn't matter. Replacing 'Cuba' with 'Canada' in a tweet should not change the prediction.

DIR, a Directional Expectation test, applies a change with a known effect. Adding 'You are lame' to a tweet cannot make the sentiment more positive.

The key point: **INV and DIR need no labels**. They only compare the model's answer before and after the change."

Slide 10 — Generating tests and experimental design

"Writing thousands of tests by hand would be impossible, so CheckList uses **templates**. In 'I {NEGATION} {POS_VERB} the {THING}', each slot is filled from a list — 'didn't' or 'can't say I', 'love' or 'like', 'food', 'flight' or 'service' — and every combination becomes a test case. The tool also suggests words with RoBERTa, and has lists of names and ready-made typos.

The evaluation has two parts.

In **part A**, they tested the models: three tasks, the commercial APIs of Microsoft, Google and Amazon, plus BERT and RoBERTa. The metric is the failure rate of each test.

In **part B**, they tested CheckList with people: a five-hour session with Microsoft's sentiment team, and a controlled experiment with **18 users**, two hours each, in three groups of six — no help, a list of capabilities, or the full tool.

So, what did they find?"

Part 4 — Results, applications and conclusion

Slide 11 — Primary results

"Thanks. There are two main results.

On the left, the **models**. Models considered 'solved' on benchmarks fail very basic things. When the negation comes at the end of the sentence, Microsoft's and Amazon's models fail **100%** of the cases. In reading comprehension, the model fails every simple 'he' versus 'she' test — **100%**. On the Quora task, where BERT beats human accuracy on the benchmark, it confuses 'before' and 'after' **98%** of the time. And given 'John is not a doctor, Mary is', the model says John is the doctor **89%** of the time — but only 3% when the names are swapped. That's a clear gender bias.

Surprisingly, the research models did better than the commercial ones on most tests. The authors think the models rely on **shortcuts** instead of real understanding.

On the right, the **users**. Compared with the group without help, users with the full tool created **twice as many tests**, had **27 times more cases per test**, and found about

three times as many serious bugs. Microsoft's team also found new bugs in a mature model.

One caveat: each group had only six people, so this is a strong indication, not statistical proof."

Slide 12 — Main applications of CheckList

"Where can CheckList be used in practice? The paper points to four applications.

Choosing a model: we can compare models skill by skill instead of by one score, as the authors did with research and commercial models.

The development cycle: the same tests can be re-run on every new version of a model. Microsoft's team said they would add CheckList to their development cycle.

Benchmarks and pipelines: the tests can be added to existing benchmarks and evaluation pipelines.

Shared test suites: teams can build and reuse tests together, since many tests work across tasks.

And all of this works on any model, even a paid API, because CheckList only needs inputs and outputs — **no access to data or weights**."

Slide 13 — Conclusion

"To wrap up: **accuracy is useful, but not enough**.

The takeaways: CheckList works for any task and any model, it finds bugs even in commercial products, and it's easy to learn.

It also has limitations. It only tests behavior, so it doesn't catch things like labelling errors or security issues. **Passing a test doesn't prove the model is correct.** And the user study was small.

If you remember one thing, make it this: CheckList changes the question from **'How accurate is the model?' to 'What can the model actually do?'**

Thank you. We're happy to take any questions."

Slide 14 — References

"These are our references: the paper itself, plus three works it builds on — Beizer for black-box testing, Segura and colleagues for metamorphic testing, and Tenney and

> colleagues for the probing result we mentioned. The CheckList tool is open source on GitHub.”

Possible questions (short answers)

Question	Answer
What is black-box testing?	Testing only through inputs and outputs, without looking at the model’s code or weights.
How can you test without labels?	INV and DIR compare the model’s answer before and after a change, not with a correct answer.
Where does the 85% come from?	BERT’s failure rate on the “negation at the end” test in Table 1 of the paper.
Why did research models beat commercial ones?	The paper reports it as a surprise and gives no definitive explanation.
Does passing a test prove a model is fair?	No. The authors say the absence of failures does not prove correctness.
Does CheckList replace other evaluations?	No, it complements them and organizes them in one framework.
What are its limitations?	It does not cover labelling errors, data versioning, security attacks or interpretability, and it depends on who writes the tests.